From statement to implementation

A worked-problem companion to the math and implementation references. Landscape, two columns, larger prose for derivations. Edition 2026-10-28.

Find a model

1 Choose and check a model **1**
- 1.1 Start with the requested object . . . 1
  - 1.1.1 A five-line design before typing . . . 1
  - 1.1.2 Terms used in the graph cases . . . 1
- 1.2 Recognition routes . . . 2
  - 1.2.1 Numbers and counting . . . 2
  - 1.2.2 Graphs and dependencies . . . 2
  - 1.2.3 Arrays, ranges, and optimization . . . 2
- 1.3 A nearby problem that needs a different tool . . . 2
  - 1.3.1 Signed exact sums: replace the window by prefix frequencies . . . 2
  - 1.3.2 Shared code for this book . . . 3

2 Arithmetic and representations **3**
- 2.1 Worked math reductions . . . 3
  - 2.1.1 Fuzzy Factorization: make the allowed error useful . . . 3
  - 2.1.2 Displaying Decimals: solve one fraction before aggregation . . . 4
  - 2.1.3 Smallest missing subset sum: protect a covered interval . . . 4
  - 2.1.4 Exact scaling, gcd, then a remainder orbit . . . 4

3 States and transitions **5**
- 3.1 Graph and DP casebook . . . 5
  - 3.1.1 Shortest route with a parity condition . . . 5
  - 3.1.2 A grid with moving hazards: CSES Monsters . . . 6
  - 3.1.3 Digit completion as a remainder automaton: 2017 Enigma . . . 7
  - 3.1.4 Weighted dominance DP with strict ties: 2017 Fundraising . . . 8

4 Change the representation **9**
- 4.1 Model and pattern cases . . . 10
  - 4.1.1 Case: intervals - Regional 2024 J, *Jigsaw of Shadows* . . . 10
  - 4.1.2 Case: flip - Masters Mexico 2022 F, *Flip* . . . 10
  - 4.1.3 Case: matching or flow - assigning work with quotas . . . 11
  - 4.1.4 Case: capacity - binary search plus a greedy proof . . . 11

5 More states and tree reductions **12**
- 5.1 Three reusable DP and tree patterns . . . 12
  - 5.1.1 Order every item once: subset Hamiltonian path . . . 12
  - 5.1.2 Pair disjoint endpoints in a tree . . . 13
  - 5.1.3 Turn descendants into a range . . . 14
  - 5.1.4 Find the kth object by skipping completion blocks . . . 14

6 Adapt, test, and practice **15**
- 6.1 A budget turns subset search into DP . . . 15
- 6.2 Turn a worked case into a new solution . . . 15
  - 6.2.1 How to move from the naive recurrence . . . 15
- 6.3 What to test before trusting the adaptation . . . 16
  - 6.3.1 Counterexamples worth writing by hand . . . 16
- 6.4 Practice without choosing another topic . . . 16
- 6.5 When the current pattern is insufficient . . . 16

Choose and check a model (1)

1.1 Start with the requested object

A statement's story is less useful than its output and constraints. Write one sentence saying what counts as a feasible answer. Then identify the quantity optimized or counted. "Find a route" can mean any route, fewest edges, smallest weight, a simple path, or a route that obeys a time limit. Those lead to different algorithms even on the same graph.

Use the code references after this choice. The math book holds formulas and numeric contracts; the implementation book holds foundational helpers and KACTL specialists. This book supplies reductions, traces, code adaptations, and failure cases. The cases marked synthetic isolate one decision; the named contest cases link their sources. None of the example input/output wrappers has been submitted to a judge. The five new helper modules have local independent-oracle tests.

1.1.1 A five-line design before typing

1. **Object.** A set, subsequence, contiguous interval, path, walk, assignment, or number? Preserve this distinction in the representation.
2. **Information.** What must be known to decide the next legal move? If two histories can have different legal continuations, they cannot share one DP or search state.
3. **Naive method.** Enumerate the choices or write the recurrence. Identify the repeated work rather than guessing an advanced structure.
4. **Reduction.** Explain why discarded information cannot change the answer. For greedy, explain why an optimal solution can use the choice. For binary search, prove feasibility stays true as the bound increases.
5. **Cost.** Count states, transitions, stored values, and numeric bounds. Multiply dimensions; a small-looking state with three indices can be too large. Compare the estimate with the actual time/memory limits.

1.1.2 Terms used in the graph cases

A vertex represents a state; an edge represents one permitted transition. An adjacency list stores each vertex's outgoing neighbors. Directed edges allow travel only in their stated direction; undirected edges are inserted both ways. A walk may repeat vertices, while a simple path may not. BFS uses a first-in, first-out queue, so it explores equal-cost transitions by increasing number of moves. A DP state stores an answer for a subproblem; its transition combines already solved states in a dependency-respecting order.

1.2 Recognition routes

A cue suggests a candidate, not a proof. Read the condition in the right column before selecting the tool. The page references lead to derivations.

1.2.1 Numbers and counting

Clue	Model and required check
Huge integer, allowed error	Keep significant digits and prove relative error. Exact output would invalidate this. Fuzzy, p. 3.
Exact decimal prices	Parse scaled integers with a common scale; gcd or divisibility then applies exactly. Scaling, p. 4.
Unknown digits and divisibility	State is position plus prefix remainder; add any other future-relevant property. Enigma, p. 7.
Repeated process on a circle	Work modulo the size. Repeated step d visits $n/\gcd(n,d)$ positions; there are $\gcd(n,d)$ orbits.
Count pairs or expected total	Fix one contribution, count how often it appears, then sum. Specify ordered/unordered and replacement first.
Synchronize periodic schedules	Solve congruences. CRT with shared factors requires residue differences divisible by the gcd.
Smallest missing subset sum	Sorted positive values may cover a whole interval. Prove coverage before skipping enumeration, p. 4.

For a contribution example, array element a_i appears in $(i+1)(n-i)$ subarrays: choose a left endpoint from $0..i$ and right endpoint from $i..n-1$. Therefore the sum of all subarray sums is $\sum_i a_i(i+1)(n-i)$. This avoids enumerating the subarrays. It does not count subarrays whose sum equals a target; that predicate couples elements and needs a different reduction, p. 2.

1.2.2 Graphs and dependencies

Clue	Model and required check
Fewest equal-cost moves	BFS on complete states, possibly position plus mask/-parity. Product states, p. 5.
Costs only zero and one	0–1 BFS; discovery is not final until distance improvements stop. Arbitrary costs need another algorithm.
Minimum nonnegative total cost	Dijkstra; add resource/phase information to vertices when future choices depend on it.
Can events be ordered?	Directed edges from prerequisite to dependent; topological order detects cycles and supports DAG DP.
Connect everything cheaply	MST, usually Kruskal with DSU; this minimizes edge total, not source-to-target distance.
Mutual directed reachability	SCC, then a condensation DAG if processing components. Undirected components are not a substitute.
Two groups of objects	Bipartite matching for one-to-one assignment; flow for quotas. Define each edge's meaning, p. 11.
Avoid spreading hazards	Earliest hazard arrivals become deadlines for a constrained player BFS, p. 6.

A tree is a connected undirected graph without cycles. Rooting gives each nonroot vertex one parent. Process parents before children for depth/root sums; reverse that order for child-to-parent DP. LCA answers path ancestry, not arbitrary range aggregates. For path sums with only static edge weights, root distances give $d(u,v) = D[u] + D[v] - 2D[\mathrm{LCA}(u,v)]$; dynamic path updates need more structure. The supplied root order is not a contiguous subtree Euler tour.

1.2.3 Arrays, ranges, and optimization

Clue	Model and required check
Static interval sum	Prefix sums and subtraction; negative values are fine.
Many range additions, one final array	Difference endpoints, then prefix scan. Interleaved queries need another structure.
Point changes, range sums	Fenwick; use delta for assignment. Prefix order statistics require nonnegative frequencies.
Associative range summary	Segment tree with a correct identity and merge; sum, min, max and ordered concatenation differ.
Overlapping coverage	Sort intervals, merge, sum lengths. Depth of coverage instead needs events, p. 10.
Minimize largest feasible bound	Prove a monotone decision predicate, then binary search; its internal solver still needs proof, p. 11.
Best earlier compatible choice	Write $dp[i] = w_i + \max dp[j]$. A dominance/prefix query may replace the scan, p. 8.
Small integer budget, optional items	DP by processed items and capacity; scan direction controls reuse, p. 15.
Few independent binary actions	Enumerate subsets if the operation count fits. Commuting toggles give a concrete example, p. 10.

1.3 A nearby problem that needs a different tool

1.3.1 Signed exact sums: replace the window by prefix frequencies

Synthetic task: count nonempty contiguous subarrays with sum K. If all values are nonnegative, expanding a window cannot decrease its sum and removing the leftmost value cannot increase it. This supports the window helper in the implementation book. Negative values break those implications: for $[3,-2]$ and $K=1$, discarding the first 3 when it looks too large loses the valid two-element subarray.

Define $P[0]=0$ and $P[r] = a_0 + \cdots + a_{r-1}$. An interval $[l,r)$ has sum K exactly when $P[l] = P[r] - K$. Scan r left to right and count earlier prefix values. Initialize frequency of 0 to one. Query before inserting $P[r]$, so $l < r$ and the empty subarray is excluded.

Trace $[1,-1,1]$, $K=1$: start with frequencies $\{0:1\}$. Prefix 1 finds one earlier 0; prefix 0 finds no earlier -1; prefix 1 now finds two earlier zeros. Total 3, from $[0,1)$, $[0,3)$ and $[2,3)$. Storing only whether a prefix exists would lose multiplicity.

The module uses an ordered map for guaranteed $O(n \log n)$ time and $O(n)$ space. A hash map changes this to expected linear time, with hash/runtime tradeoffs. Prefix sums and $P[r] - K$ must fit the numeric type. For minimum length instead of count, keep the latest matching index; for maximum length, keep the earliest. For arbitrary intervals with inequalities, an equality map alone is insufficient; consider ordered prefix queries.

1.3.2 Shared code for this book

The new modules use namespace `finals_cases`. Copy their shared prelude and the listed dependencies once. `Fundraising.h` additionally uses the implementation book's segment tree; `CapacitySearch.h` uses its binary search. The includes below point to the repository files; when typing into one contest file, paste those dependencies instead of retaining local paths. Existing helpers keep `finals_core` or `finals_math`; qualify a call or use the corresponding namespace.

Prelude.h

Description: Shared includes, aliases and bounds. Include before the modules.

"../../content/finals-implementations/BinarySearch.h", "../../content/finals-implementations/SegmentTree.h", <array>, <map>, <optional>, <string> f0e086, 5 lines

```
namespace finals_cases {
using namespace std;
using ll = long long;

} // namespace finals_cases
```

ExactSum.h

Description: Count exact subarray sums with signed entries.

"Prelude.h" bc4a1f, 15 lines

```
namespace finals_cases {
// Prefix sums, prefix-target, counts and answer must fit ll.
inline ll count_exact_sum(const vector<ll> &a, ll target) {
  map<ll, ll> freq;
  freq[0] = 1;
  ll prefix = 0, answer = 0;
  for (ll x : a) {
    prefix += x;
    auto it = freq.find(prefix - target);
    if (it != freq.end()) answer += it->second;
    ++freq[prefix];
  }
  return answer;
}
} // namespace finals_cases
```

Practice variant: count zero-sum nonempty intervals in $[0, 0]$. Answer: 3. The three prefixes are all zero; each new prefix pairs with every earlier one. Inserting before querying would incorrectly count an empty interval at each step.

Arithmetic and representations (2)

2.1 Worked math reductions

These cases practice the step between recognizing a topic and knowing what to implement. Each starts with the requested output, keeps a tempting wrong approach visible, then gives a small trace and a route through the math helpers. The walkthroughs explain how to reason about the named tasks; they are not claims that one formula fits every nearby problem.

2.1.1 Fuzzy Factorization: make the allowed error useful

Recognition. The input is a decimal integer with as many as 1,000 digits, but the answer may differ by relative error at most 10^{-9}. That permission changes the problem: the goal is not to factor the input exactly. Look for a nearby integer whose factorization is easy, and prove its error against the original denominator X.

Reduction. Keep the first ten decimal digits. Write $X = m10^e + r$, where e is the number of omitted digits and $0 \le r < 10^e$. Choose $Y = m10^e$. If X has more than ten digits, then $m \ge 10^9$, so

$$\frac{|X - Y|}{X} = \frac{r}{X} < \frac{10^e}{m10^e} = \frac{1}{m} \le 10^{-9}.$$

The inequality is strict because the discarded suffix is smaller than 10^e. If there are at most ten digits, choose $Y = X$ exactly. Thus only m, at most $10^{10} - 1$, needs factorization; the appended power of ten contributes known factors.

A tempting wrong approach is converting the thousand-digit token to floating point and rounding it. That loses the suffix unpredictably, so neither the selected integer nor its error bound is controlled. Another wrong turn is factoring all of X: exact factorization is unnecessary. Truncating to fewer than ten significant digits without recomputing the bound also does not establish the tolerance.

Trace. For $X = 1234567890123$, keep $m = 1234567890$, omit $e = 3$ digits, and take $Y = 1234567890000$. The discarded suffix is $r = 123$, so $r/X < 10^{-9}$. Since $m = 2 \cdot 3^2 \cdot 5 \cdot 3607 \cdot 3803$ and $10^3 = 2^3 5^3$, combine exponents to obtain $Y = 2^4 \cdot 3^2 \cdot 5^4 \cdot 3607 \cdot 3803$. This combining step matters: output each distinct prime once with its full exponent.

Implementation sequence. Read the token as text. Call `finals_math::fuzzy_factors(s)`: it validates positive canonical decimal input, reads the prefix without overflow, trial-factors m, then adds e to the powers of 2 and 5 in a map. Emit that map in the statement's format. The helper returns an empty map for 1, so adapt that boundary if a variant permits $X = 1$; the 2025 task has $X \ge 2$. Work is $O(|s| + \sqrt{m})$ with $\sqrt{m} < 10^5$; output size is the number of distinct factors. It does not solve exact factorization of a huge integer, a different tolerance without a new prefix bound, or a task asking for X's exact factors. 2025 F statement gives $X \le 10^{1000}$ and permits output primes through 10^{18}.

Practice variant. With relative error 10^{-6}, how many leading digits suffice? Six give $m \geq 10^5$ and error $< 10^{-5}$, which is insufficient. Seven give $m \geq 10^6$ and strict error $< 10^{-6}$. The threshold is one more retained digit than the exponent in the tolerance.

2.1.2 Displaying Decimals: solve one fraction before aggregation

Recognition. The output asks for a display length, so separate integer part from fractional expansion. A reduced denominator containing only 2s and 5s terminates; a remaining coprime factor creates a recurring remainder cycle. This reduces one ratio. The 2025 problem averages this value over every ordered pair of currencies, a separate aggregation problem.

Reduction. For nonnegative a and positive b, reduce the fraction's denominator to $q = b/\gcd(a, b)$. Write $q = 2^u 5^v r$ with $\gcd(r, 10) = 1$. Long division needs $\max(u, v)$ digits before the remainder repeats. Then the least positive k with $10^k \equiv 1 \pmod{r}$ is the period; for $r = 1$ there is no recurring part. The complete length is

$$\text{digits}(\lfloor a/b \rfloor) + \max(u, v) + \text{ord}_r(10),$$

with order zero when $r = 1$. The integer-part term counts the zero before the decimal point for a proper fraction, as the statement requires.

A tempting mistake is using $\phi(r)$ as the period. The order only divides $\phi(r)$ and can be smaller. Another is ignoring the factor valuations: the prefix uses their maximum, while the leftover coprime denominator controls the cycle. For a terminating fraction, do not invent a recurring digit.

Trace. Take 1/28. Since $28 = 2^2 \cdot 7$, the nonrepeating prefix has length 2 and residual denominator 7. Multiplying successive remainders by 10 modulo 7 gives $1 \to 3 \to 2 \to 6 \to 4 \to 5 \to 1$, a cycle of length 6. The display is $0.03\overline{571428}$: one leading zero, two prefix digits, and six recurring digits, for 9 digits total. Here "fractional length" means digits after the point; the helper returns integer-part digits separately.

Implementation sequence. Call `finals_math::decimal_shape(a,b)` for one nonnegative fraction. It returns the integer-part digit count, nonrepeating fractional length, and period. It reduces by gcd, strips 2s and 5s, then calls `finals_math::order_mod(10,r)` when $r > 1$. That routine starts at $\phi(r)$ and divides out prime factors while the modular power remains 1. Trial factorization is part of the cost, so respect the helper's bound.

Do not mistake this component for a complete solution to 2025 D. The contest asks for an expectation across N^2 ordered pairs, with $N \leq 10^5$ and values at most 10^5; calling this helper per pair is quadratic. Grouping equal values into gcd classes is one possible direction for aggregation, but this walkthrough does not give or establish a full-limit algorithm. See the 2025 D statement.

Practice variant. For 5/14, the denominator is $2 \cdot 7$, giving one prefix digit and $\text{ord}_7(10) = 6$; there are seven fractional digits. The integer part is zero, counted as one digit for the complete-display convention from 2025 D.

2.1.3 Smallest missing subset sum: protect a covered interval

Recognition. The task asks for the smallest positive integer that no subset can sum to. The list may have a million positive values, but no target bound makes a subset-sum table attractive. Ask whether reachable sums have a simple shape after sorting. Positivity lets us extend a contiguous interval without creating holes.

Reduction and proof. Sort the values and maintain R such that every sum in $[0, R]$ is reachable. Initially only 0 is covered, so $R = 0$. For the next value x, sums not using it cover $[0, R]$; sums using it cover $[x, x + R]$. If $x \leq R + 1$, these ranges touch or overlap, and all sums through $R + x$ become reachable. Update $R \leftarrow R + x$. If $x > R + 1$, the processed positive values sum to R, and every remaining value is at least x; thus $R + 1$ cannot be formed and is the answer.

A tempting approach is returning the smallest absent element from a set of enumerated subset sums. That set can grow exponentially and still misses the proof that all smaller sums are covered. Another error is allowing zero or negative values without revisiting the invariant: the proof uses positive inputs.

Trace. For $1, 1, 3, 4$, begin with $R = 0$. The first 1 is at most $R + 1 = 1$, so cover through 1. The second 1 is at most 2, so cover through 2. The value 3 meets the boundary and extends coverage through 5. The 4 extends it through 9. Every sum from 0 to 9 exists; the smallest missing positive sum is 10. Had the third value been 4, the first gap would have been 3, and later values could not fill it.

Implementation sequence. Sort, set `next=1`, then for each x: if $x >$ `next`, stop; otherwise add x to `next`. This variable is always $R + 1$, so it is the answer both at the first gap and after the final element. Use a 64-bit accumulator: with 10^6 values each at most 10^6, the total may reach 10^{12}. Time is $O(n \log n)$; extra space depends on whether sorting may modify the input. The math chapter's greedy snippet implements this invariant.

If zero is permitted, skip it because it does not extend the interval. With negatives the positive-prefix method no longer applies. If the task asks for the number of subsets producing each sum, this proof does not count multiplicities. Masters Mexico 2022 H statement sets positive values through 10^6 and $n \leq 10^6$.

Practice variant. For sorted values $1, 2, 2, 9$, R advances from 0 to 1, to 3, to 5; then 9 exceeds $R + 1 = 6$. Answer: 6.

2.1.4 Exact scaling, gcd, then a remainder orbit

Recognition. This synthetic task combines exact input with the decimal-cycle reasoning above: two prices arrive as decimal strings; find how many digits their exact ratio needs, including its first full recurring period. Decimal text suggests scaling both values by the same power of ten. That common scale cancels in a ratio, gcd removes shared factors, and modular remainders reveal the period.

Reduction. With three fractional places, parse each string as an integer number of thousandths. For example, 0.750 and 2.100 become 750 and 2100. The ratio is unchanged by their shared scale; $\gcd(750, 2100) = 150$ reduces it to $5/14$. Since $14 = 2 \cdot 7$, there is one nonrepeating decimal place, then an orbit modulo 7. The orbit under $r \mapsto 10r \bmod 7$ has six distinct remainders, so the fractional part uses seven digits. No binary floating-point rounding enters the gcd or cycle.

A tempting mistake is parsing both strings as `double` and rounding their quotient to infer a period. Approximate equality cannot establish an exact remainder cycle. Another is applying different scales to numerator and denominator without normalizing them: the scale must be consistent. If the inputs have different fractional lengths, normalize to a shared decimal scale first.

Trace. Long division of $5/14$ begins with remainder 5. The first digit is 3, leaving remainder 8; recurring digits follow remainders $8, 10, 2, 6, 4, 12$, then remainder 8 returns. Multiplication by 10 modulo 14 generates this orbit; stripping its factor 2 explains why the period is the order of 10 modulo 7. A repeated remainder proves all following digits repeat because division is deterministic.

Implementation sequence. Parse both strings with `finals_math::parse_fixed(s,3)`, which rejects excess fractional digits and returns an exact signed 64-bit scaled integer. Confirm the prices are nonnegative and the denominator positive, then call `finals_math::decimal_shape(a,b)`. For 750/2100 it returns integer digits 1, nonrepeating length 1, and period 6. Combine these according to the task's display convention. Parsing costs linear time in string length; gcd is logarithmic, and period computation factors $\phi(r)$ by trial division. Check that scaled values and products fit the helper's 64-bit contract. For arbitrary-precision prices, use an arbitrary-precision route; this helper does not promise it.

The adaptation boundary is the requested output. If only the recurring period is requested, return 6 here instead of adding the prefix. A task asking for rounded decimal value needs a rounding rule and precision analysis. If the denominator has only factors 2 and 5, period is zero. 2025 D statement defines its display convention and includes individual-ratio examples.

Practice variant. For 0.125/0.375, scaling to thousandths gives $125/375 = 1/3$. The prefix is empty and the period is 1 because $10 \equiv 1 \pmod 3$. The ratio is unchanged by extra trailing zeroes. If those exceed the parser's chosen precision, strip them or increase the common scale before parsing.

States and transitions (3)

3.1 Graph and DP casebook

These cases practice a repeatable decision: identify what the answer depends on, choose the smallest state that preserves that information, and only then choose a traversal or data structure. The reasoning appears in prose and short pseudocode, with links to tested local helpers where one matches the problem contract. A helper is a reusable implementation component, not a complete submission.

3.1.1 Shortest route with a parity condition

This is a synthetic graph example.

Suppose an undirected graph has unit-cost edges, and the task asks whether a walk from s to t uses an even number of edges, or asks for the shortest such walk. A plain BFS on vertices looks tempting: it gives the shortest distance to every vertex. But knowing that a vertex is reachable does not tell us whether it is reachable after an even or odd number of steps. If a vertex is reached by paths of lengths two and three, those histories have different eligibility for the destination condition. Keeping only one distance per vertex throws away relevant information.

Use a product graph whose state is (v, p), where v is the current vertex and $p \in \{0, 1\}$ is the number of traversed edges modulo two. Each original edge (u, v) creates transitions $(u, p) \to (v, 1 - p)$ and $(v, p) \to (u, 1 - p)$. The invariant is: BFS distance to (v, p) is the fewest edges in any walk from s to v whose length has parity p. Initialize only $(s, 0)$ at distance zero. Then a shortest even walk to t is the distance at $(t, 0)$; an unreachable state remains absent. If actual reconstruction is required, retain a parent state for each visited product state and walk backward from $(t, 0)$.

Pseudocode:

```
dist[n][2] = UNREACHED
dist[s][0] = 0; queue.push((s, 0))
while queue not empty:
    (u, p) = queue.pop_front()
    for v in graph[u]:
        q = 1 - p
        if dist[v][q] is UNREACHED:
            dist[v][q] = dist[u][p] + 1
            parent[v][q] = (u, p)
            queue.push((v, q))
```

The expanded graph has $2n$ states and $O(m)$ transitions, so time and memory stay $O(n + m)$. For ordinary reachability, the local `bfs(g,{source})` helper is enough. For parity, represent (v, p) as `2*v+p` and construct the implicit transitions while traversing, or build a graph with $2n$ vertices and call the helper. Its returned parent array then supports `restore_path` on expanded state IDs; decode each ID by division and remainder by two. A direct implementation may store two distances per vertex instead.

Tiny trace: let the graph be the triangle 0–1–2–0, with $s = 0$. The initial state is $(0, 0)$. One BFS layer discovers $(1, 1)$ and $(2, 1)$. The next layer discovers $(2, 0)$ through 1 and $(1, 0)$ through 2. Thus the shortest even walk from 0 to 2 has length 2, namely $0, 1, 2$, even though the shortest unrestricted walk has length 1. This illustrates why "visited vertex" is not a sufficient marker.

The model changes if edges carry non-unit costs: parity still belongs in the state, but BFS no longer minimizes cost. Use Dijkstra on the product graph for nonnegative costs, or the appropriate weighted shortest-path method if negative edges are allowed. If the condition is "exactly k edges," a modulo-two state is insufficient; use a step dimension when k is small, or a method suited to the bound. If the question asks for a simple path, the expanded walk can revisit an original vertex with a different parity, so the walk-based model alone does not enforce simplicity.

Practice variant: find the shortest walk whose length is congruent to 1 modulo 3. Answer: use state (v, p) with $p \in \{0, 1, 2\}$; every edge moves to $(v, (p + 1) \bmod 3)$; start at $(s, 0)$ and read $(t, 1)$. The product graph has $3n$ states. This generalization works because the condition depends only on the edge count modulo 3.

3.1.2 A grid with moving hazards: CSES Monsters

In this grid task the player and every monster can move one cell per turn; the player must reach any boundary cell without ever occupying a cell with a monster. A single BFS from the player is the obvious first attempt, but it answers the wrong question. A cell that is reachable in three steps might already be occupied by a monster at time two. Treating monsters as static walls ignores the timing constraint; moving them in the same queue as the player also mixes two different arrival-time questions.

The sufficient state has two parts conceptually: the player's cell and the time at which the player arrives. Rather than include time in a large time-expanded graph, first compute the earliest monster arrival time $M[x]$ for every cell using one multi-source BFS from all monster cells. Then compute a player BFS from A, entering a neighbor y only if the player's arrival time $P[x] + 1$ is strictly less than $M[y]$. Equality is unsafe because both arrive during the same turn. The invariant for the second BFS is that each queued cell is reached by a valid player path and at its shortest valid time; all such paths are safe because every earlier arrival along that path passed the same strict test.

Steps:

1. Run multi-source grid BFS from every monster; walls are impassable. Unreached cells have $M = \infty$.
2. Start player BFS at A with $P[A] = 0$. For each open neighbor y, accept it only when it is unvisited and $P[x] + 1 < M[y]$.
3. Stop when a valid boundary cell is dequeued. Store its parent cell and move direction at discovery time; reverse those moves to print a path. If no boundary is reached, report failure.

HazardEscape.h

Description: BFS with hostile arrival deadlines.

"Prelude.h" 0f715b, 69 lines

```
namespace finals_cases {
// Exactly one A; rectangular . # A M grid; flattened size fits
// int.
inline optional<string>
escape_monsters(const vector<string> &a) {
  if (a.empty() || a[0].empty()) return nullopt;
  int h = int(a.size()), w = int(a[0].size()), n = h * w;
  for (auto &row : a) assert(int(row.size()) == w);
  const int dr[] = {-1, 1, 0, 0}, dc[] = {0, 0, -1, 1};
  const string moves = "UDLR";
  vector<int> danger(n, -1), dist(n, -1), parent(n, -1),
      via(n);
  queue<int> q;
  int start = -1, starts = 0;
  for (int r = 0; r < h; ++r)
    for (int c = 0; c < w; ++c) {
      int u = r * w + c;
      if (a[r][c] == 'M') {
        danger[u] = 0;
        q.push(u);
      }
      if (a[r][c] == 'A') {
        start = u;
        ++starts;
      }
    }
  assert(starts == 1);
  auto neighbor = [&](int u, int d) {
    int r = u / w + dr[d], c = u % w + dc[d];
    if (r < 0 || r >= h || c < 0 || c >= w || a[r][c] == '#')
      return -1;
    return r * w + c;
  };
  while (!q.empty()) {
    int u = q.front();
    q.pop();
    for (int d = 0; d < 4; ++d) {
      int v = neighbor(u, d);
      if (v < 0 || danger[v] != -1) continue;
      danger[v] = danger[u] + 1;
      q.push(v);
    }
  }
  dist[start] = 0;
  q.push(start);
  while (!q.empty()) {
    int u = q.front();
    q.pop();
    int r = u / w, c = u % w;
    if (r == 0 || r == h - 1 || c == 0 || c == w - 1) {
      string path;
      for (int v = u; v != start; v = parent[v])
        path += moves[via[v]];
      reverse(path.begin(), path.end());
      return path;
```

```
    }
    for (int d = 0; d < 4; ++d) {
      int v = neighbor(u, d), t = dist[u] + 1;
      if (v < 0 || dist[v] != -1) continue;
      if (danger[v] != -1 && t >= danger[v]) continue;
      dist[v] = t;
      parent[v] = u;
      via[v] = d;
      q.push(v);
    }
  }
  return nullopt;
}
} // namespace finals_cases
```

The tested `finals_cases::escape_monsters(board)` helper packages these steps and returns an optional UDLR path; an empty optional means no escape. It accepts the statement's `.`, `#`, `A`, and `M` alphabet. Use the returned path directly when one valid route is enough.

This is $O(hw)$ time and space for an h by w board, since each cell is processed a constant number of times. The local `grid_bfs(board,sources,'#')` helper computes ordinary grid distances with four-neighbor motion; call it once for monsters and once for the player only if you can apply the safety filter during player expansion. A plain call for player distances followed by a check of only the final boundary is incorrect, because an unsafe intermediate cell may have been traversed. For that constrained BFS, reuse the helper's neighbor and parent pattern while adding the arrival test. Flatten cells with `id = row*w + col` and decode with division and remainder by `w`. The statement permits up to 1000×1000 cells, so the product and all stored IDs must fit the chosen integer type.

Tiny trace: consider rows `#######`, `#.....#`, `#M.A..#`, `#.....#`, and `#####.#`. The player starts at row 2, column 3. The left neighbor has monster arrival time 1, so entering it at time 1 fails the strict test. A shortest safe route goes down, right, right, then down to the only open boundary cell: it takes 4 steps. The monster at $(2, 1)$ needs 6 steps to reach that exit at $(4, 5)$, so the player arrives safely. This immediate comparison prevents the mistake of declaring an escape merely because a boundary is geometrically reachable.

Boundary details matter: if the player begins on a boundary, the zero-step path is valid before any hazard can arrive, so check the start as an exit. Unreachable monster distances behave as infinity; do not accidentally interpret a default zero as an arrival at time zero. Monsters can be initialized together because all spread at the same speed. If hazards have different speeds, doors, or time-dependent routes, a single unweighted multi-source BFS may no longer represent earliest arrival; use the matching weighted or time-dependent model. If the player has a different speed, scale times consistently or compare arrival times in common units.

Practice variant: hazards move twice as fast as the player, and all moves remain on unit grid edges. Answer: monster BFS gives edge distance d_M; the hazard time is $d_M/2$, so compare player arrival d_P by cross multiplying $2d_P < d_M$ (assuming positions change at half-turn granularity). If turns are indivisible and the rules specify simultaneous discrete moves, derive the exact timing convention from the statement before applying this formula.

Source: the primary CSES Monsters statement specifies simultaneous moves, adversarial knowledge of the path, and a boundary escape. The two-BFS method above is the state reduction used to enforce those rules.

3.1.3 Digit completion as a remainder automaton: 2017 Enigma

Enigma gives a fixed-length decimal pattern containing digits and question marks, and asks for its smallest completion divisible by N, with no leading zero. A tempting approach fills unknown digits with their smallest allowed values and tries to repair divisibility later. This fails because a locally small digit can leave a suffix remainder that no legal completion can fix. Enumerating all replacements is exponential in the number of question marks. The key observation is that a prefix's entire numeric value is unnecessary: future divisibility depends only on its remainder modulo N.

Define $F[i][r]$ to mean: starting before pattern position i with current prefix remainder r, there exists a legal suffix completion that ends with remainder zero. The recurrence tries each allowed digit d and sets $F[i][r]$ if $F[i+1][(10r + d) \bmod N]$ is true. Base case: $F[L][0] =$ true and $F[L][r] =$ false for $r \neq 0$. At position zero, disallow digit zero. To reconstruct the lexicographically smallest number, start with remainder zero and choose the smallest allowed digit whose next state is feasible in the next row. This greedy choice is safe because the table certifies that some valid suffix remains.

The local `finals_dp::complete_digits(pattern,modulus)` implements this API directly: it returns an optional smallest string, forbids a leading zero by default (including a one-character zero), and runs in $O(10LN)$ time and $O(LN)$ memory. Its boolean table is filled backward; reconstruction walks forward. For a larger modulus, this memory bound may be the limiting factor, so check constraints before allocating $(L + 1)N$ entries. The official 2017 statement has $L \leq 1000$ and $N \leq 1000$, making the table the intended shape.

Tiny trace: pattern `?2?` and modulus 7. The smallest legal first digit is 1. After choosing 1 the prefix remainder is 1. At position 1 the fixed digit 2 changes it to $(10 \cdot 1 + 2) \bmod 7 = 5$. The last digit d must satisfy $(10 \cdot 5 + d) \bmod 7 = (1 + d) \bmod 7 = 0$, so $d = 6$. The answer is 126. Trying first digit 0 is forbidden; choosing 1 is justified by suffix feasibility, not by guessing.

The official statement allows the initial character to be a question mark but requires the completed number not to start with zero; its samples also include an impossible pattern. Keep "no completion" distinct from a valid answer containing zero. A common implementation bug is to compute the forward remainder correctly but use a suffix table with the wrong meaning; state the table meaning before writing the recurrence. Another is to use `10*r+d` in a narrow integer type when bounds could overflow; the local helper widens before multiplication.

Modification: if the task asks for the number of completions rather than the smallest one, replace boolean feasibility with counts and sum over allowed digits. The local helper does not do that; moreover, counts may need a modulus or arbitrary precision. If the task asks for the smallest number at most a bound, add a tight-to-bound state as well as the remainder. The remainder alone is no longer sufficient because the bound depends on the prefix relation to the limit.

Practice variant: complete `?3` to a multiple of 4. Answer: no completion exists, because every multiple of 4 is even while the fixed final digit is 3. The feasibility table reports that directly; there is no candidate digit or retraction to apply.

Source: the primary 2017 Enigma statement gives the pattern, divisibility, leading-zero rules, and bounds. The remainder-DP explanation matches the tested local helper.

3.1.4 Weighted dominance DP with strict ties: 2017 Fundraising

Each candidate has beauty B, fortune F, and donation D. Two people can coexist only when they are equal in both attributes or one is strictly larger in both. We seek the maximum donation sum. Sorting by one coordinate and taking a weighted LIS is close, but a naive implementation can admit illegal ties: processing points with equal fortune one at a time may let a later point use an earlier one, even though equal fortune does not satisfy the strict increase rule. A prefix sum Fenwick tree is also the wrong query: we need the best predecessor value among compatible points, not the sum of all their values.

First merge identical (B, F) pairs and sum their donations. It is safe and necessary: equal-coordinate people do not argue and can all be invited, so they act as one point of combined weight. Then sort by fortune ascending and, for equal fortune, beauty descending. For a point i, its DP value is
$D_i + \max(0, \text{best donation sum ending at a point with } F < F_i, B < B_i)$. The zero option starts a new chain. The best valid guest set forms a chain under strict increase in both coordinates: any pair in the chain can coexist, and the statement's compatibility condition excludes incomparable unequal pairs.

Compress beauty values to zero-based ranks $0..K-1$. A prefix-maximum Fenwick tree can answer the maximum DP among beauty ranks strictly below the current rank, then update the current rank with the candidate DP. If only the generic local `SegTree<T,Op>` is available, use max with identity zero (donations are positive), query half-open ranks $[0, rank)$, and point-set the rank to the maximum of its current value and the new DP. Do not use a range sum or the ordinary additive `Fenwick` helper: neither represents prefix maximum. Also, point-set must preserve the previous maximum if another valid chain ends at the same rank.

There are two correct tie strategies. One is to sort by fortune ascending and beauty descending; a sequential strict-prefix query cannot see equal beauty, and descending beauty prevents equal-fortune points from increasing in beauty. Alternatively, process all equal-fortune points as a batch: query all of them against the tree before applying any updates. Batching is often easier to reason about and remains correct with another tie sort. Use one strategy deliberately, not half of each.

The tested `finals_cases::max_donation_chain(donors)` helper in this booklet applies the batched strict-coordinate method. Its `Donor` fields are `wealth`, `beauty`, and `donation`; nonnegative donations are supported, and identical coordinate pairs are combined.

Fundraising.h

Description: Strict two-dimensional donation chains.

"Prelude.h" 0376c4, 49 lines

```
namespace finals_cases {
struct Donor {
  ll wealth, beauty, donation;
};
struct Maximum {
  ll operator()(ll a, ll b) const { return max(a, b); }
};
// Donations >=0; totals fit ll. Equal pairs contribute
// together.
inline ll max_donation_chain(vector<Donor> a) {
  sort(a.begin(), a.end(), [](Donor x, Donor y) {
    return pair{x.wealth, x.beauty} < pair{y.wealth, y.beauty};
  });
  vector<Donor> b;
  for (auto p : a) {
    assert(p.donation >= 0);
    if (!b.empty() && b.back().wealth == p.wealth &&
        b.back().beauty == p.beauty)
      b.back().donation += p.donation;
    else
      b.push_back(p);
  }
  vector<ll> ys;
  for (auto p : b) ys.push_back(p.beauty);
  sort(ys.begin(), ys.end());
  ys.erase(unique(ys.begin(), ys.end()), ys.end());
  finals_core::SegTree<ll, Maximum> tree(
      vector<ll>(ys.size(), 0), 0);
  ll answer = 0;
  for (int l = 0; l < int(b.size());) {
    int r = l;
    while (r < int(b.size()) && b[r].wealth == b[l].wealth)
      ++r;
    vector<pair<int, ll>> pending;
    for (int i = l; i < r; ++i) {
      int y =
          int(lower_bound(ys.begin(), ys.end(), b[i].beauty) -
              ys.begin());
      ll value = tree.query(0, y) + b[i].donation;
      pending.push_back({y, value});
      answer = max(answer, value);
    }
    for (auto [y, value] : pending)
      tree.set(y, max(tree.query(y, y + 1), value));
    l = r;
  }
  return answer;
}
} // namespace finals_cases
```

Use it when the input matches that contract; keep the recurrence below as the explanation for why the data-structure query is correct.

Pseudocode for the editorial ordering:

```
merge donations for each identical (beauty, fortune)
sort by (fortune ascending, beauty descending)
compress all beauty coordinates
for (B, F, D) in sorted points:
    r = rank(B)
    best = prefix_max(0, r)          // zero-based half-open ranks [0,r)
    dp = best + D
    update_prefix_max(r, dp)
answer = maximum dp (or tree-wide maximum)
```

The local segment tree uses zero-based ranks and half-open ranges, so query $[0, r)$. A Fenwick implementation may instead use one-based update indices; then query the first r entries. The local Fenwick `prefix(r)` API also means $[0, r)$. Define the index convention and prefix API together so the exclusion of equal beauty is visible in code. Complexity is $O(N \log N)$ time and $O(N)$ space, with $N \le 10^5$; donation sums require 64-bit integers because each donation may be as large as 10^9.

Tiny trace: points $(B, F, D) = (1, 1, 4), (2, 1, 7), (3, 2, 5)$. Sort at fortune 1 by beauty descending: $(2, 1, 7)$ then $(1, 1, 4)$. Each starts its own chain because neither can see an earlier strictly smaller beauty at the same fortune. At $(3, 2, 5)$, the prefix query sees both prior values and returns 7, giving 12. A fortune-ascending, beauty-ascending sort would process $(1, 1, 4)$ before $(2, 1, 7)$ and incorrectly chain them for 11, then report 16. Merging equal points matters too: two guests at $(2, 1)$ with donations 3 and 8 may both be invited, so their grouped weight is 11.

The maximum query uses identity zero because the statement's donations are positive and starting a new chain is allowed. If negative donations were permitted but the guest list had to be nonempty, initialize the final answer to negative infinity; for each point, set $dp = D + \max(0, \text{best predecessor})$, so zero starts a new nonempty chain at that point. Do not initialize the answer to zero, which would represent an invalid empty guest list when every donation is negative. Also, merging identical coordinate pairs by summing weights relies on the original positive donations: with negative donations, one could invite only some people in that group, so retain their individual choices. If the problem changed to allow equality in beauty, query through the current rank; if it allowed equality in fortune, equal-F points could interact and the descending-tie trick would no longer enforce strictness in that coordinate. For reconstruction, store the chosen predecessor point whenever a new maximum is written; ties can use any predecessor meeting the constraints.

Practice variant: require beauty strictly increasing but permit equal fortune. Answer: keep fortune ascending, and query beauty ranks strictly below as before; equal-fortune points may now chain, so process them in ascending beauty order and allow each update to affect the next point. This change is valid only because the modified rule explicitly permits equal fortune.

Source: the primary 2017 Fundraising statement defines compatibility, donation bounds, and input size. The casebook's recurrence and tie strategies are checked against the tested local helper; see `handbooks/research/graph-dp-pattern-sources.md` for source-access notes.

Change the representation (4)

4.1 Model and pattern cases

These cases train the step before coding: translate the story into objects, states, and constraints, then choose the smallest tool that preserves what the answer needs. The first two cases are real contest problems with links to their statements and official solution material. The last two are synthetic models designed to expose reusable reductions.

4.1.1 Case: intervals - Regional 2024 J, *Jigsaw of Shadows*

The statement gives an angle θ, then N flatlanders at distinct road positions X_i, each of height H_i. Each casts a shadow eastward; report the total road length covered, counting overlap once. The input bound is $N \le 10^5$, $10 \le \theta \le 80$ degrees, $X_i \le 3 \cdot 10^5$, $H_i \le 1000$, and output error may be 10^{-4}. See the official statement and official solution sketches.

The story looks like geometry, but the requested object is one-dimensional length. A right triangle gives shadow length $L_i = H_i \cot(\theta)$, so each person contributes the closed interval $[X_i, X_i + L_i]$. Once represented this way, none of the other people interact geometrically: only interval overlap matters. This is a useful recognition cue for rays, projected objects, coverage windows, and occupied time ranges: derive each range, then ask whether the answer is the union, intersection, count, or maximum depth.

Convert degrees before calling trigonometry: `rad = theta * acos(-1.0) / 180.0` and `L = H * cos(rad) / sin(rad)`. Build the vector of endpoint pairs and call `finals_core::merge_intervals<double>(intervals)`. That helper sorts by left then right endpoint, skips zero-length intervals, and merges overlap or touching endpoints. Sum `r-l` over the returned disjoint intervals. The exact helper returns intervals rather than their measure, so it can also support later queries about gaps or merged components.

Synthetic trace with $\theta = 60°$: $(X, H) = (50, 150), (0, 100), (100, 200)$. Their lengths are about 86.6025, 57.7350, and 115.4701. After sorting, the ranges are $[0, 57.7350]$, $[50, 136.6025]$, and $[100, 215.4701]$. Each overlaps the current covered prefix, so the merged range is $[0, 215.4701]$ and the answer is about 215.47. Summing raw shadow lengths counts the shared road more than once.

Sorting dominates at $O(N \log N)$ time; merging and summing use $O(N)$ time and memory. A sweep over compressed real endpoints can also work, but each compressed gap must retain its original length. Counting compressed indices would lose geometry: integer road positions do not imply integer shadow endpoints. A sweep over start/end events also works, but adds event ordering and active-count logic when a single sorted merge is enough. If the task changes to count how many shadows cover each query point, retain the event sweep or a sorted endpoint structure. If it asks for the uncovered gaps inside a finite road, clip first and keep the merged ranges. The sample allows floating output; keep the computation in double and do not round individual endpoints before merging.

Practice: two shadows are $[2, 7]$ and $[7, 9]$. What is the union length? Answer: 7, because the touching point contributes no length twice; the helper merges them to $[2, 9]$.

4.1.2 Case: flip - Masters Mexico 2022 F, *Flip*

The real problem starts from an all-white 3×3 board. Clicking a cell toggles it and its orthogonal neighbors that exist; find the minimum clicks for a given target. The statement guarantees a solution and has at most 50 boards. Read the official problem book, page 9 (problem F). This is a compact instance of a broader recognition rule: if an action is binary, its effects are fixed, actions commute, and the number of actions is tiny, enumerate action subsets before reaching for search or linear algebra.

Number cells row-major from 0 to 8. Represent a board as a 9-bit unsigned mask, with bit $r \cdot 3 + c$ set when that cell is black. For each cell i, precompute an effect mask containing i and its in-bounds up/down/left/right neighbors. A press subset p changes state by XOR with the XOR of all selected effect masks. Since each click toggles and clicks commute, pressing one cell twice cancels; therefore some minimum solution uses each cell zero or one times. Every candidate is one of $2^9 = 512$ masks. Count its set bits and keep the least count among candidates whose combined effect equals the target.

The implementation module provides `finals_cases::min_toggle_count(effects, start, target)`. Here `effects[i]` is the mask toggled by action i; `start` and `target` use the same bit layout. It returns the minimum number of actions or -1 if unreachable, with a maximum of 20 actions. For this problem use start mask zero. Parse each row by shifting left or setting bit $3r + c$ for each `*`, and construct effects using bounds checks rather than wrapping neighbors across a row.

ToggleSubsets.h

Description: Binary choices with commuting toggles.

"Prelude.h" 8e6f15, 16 lines

```
namespace finals_cases {
// k<=20 binary effects, represented in unsigned bits.
inline int min_toggle_count(const vector<unsigned> &effects,
                            unsigned start, unsigned target) {
  int k = int(effects.size()), answer = k + 1;
  assert(k <= 20);
  for (unsigned s = 0; s < (1u << k); ++s) {
    unsigned state = start;
    for (int i = 0; i < k; ++i)
      if (s >> i & 1u) state ^= effects[i];
    if (state == target)
      answer = min(answer, __builtin_popcount(s));
  }
  return answer == k + 1 ? -1 : answer;
}
} // namespace finals_cases
```

Synthetic trace:

```
**.
*..
...
```

The target mask has bits $0, 1, 3$ set. Pressing cell 0 toggles exactly cells $0, 1, 3$, so this subset has size 1 and reaches the target. No zero-click subset can produce a nonzero target, so the minimum is 1. The next sample board can be tested by the same loop; the routine does not rely on visual symmetry or board rotation.

The cost is $O(2^k \cdot k)$ time and $O(k)$ effect storage for k actions; the stated helper limit is $k \le 20$. Exhaustive enumeration is clearer and more reliable here than a generic graph shortest path, because the cost is the number of chosen independent actions and each subset has one direct result. Gaussian elimination over $\mathbb{F}_2$ can find a solution for a larger linear system, but it does not by itself minimize the number of set variables; minimum-weight solutions require extra work. If the board grows to a narrow strip, a row-mask DP may exploit width. If actions have different costs or dependencies, subset enumeration can still work at small k, but the objective or legal-subset test must change. Do not use a bit-shift mask when k exceeds the width of its integer type; use multiple words or a different representation.

Practice: two actions toggle masks 0011 and 0110, and the target is 0101 from zero. Is it reachable? Yes. Pressing both actions produces $0011 \texttt{ XOR } 0110 = 0101$, so the minimum is two. Effects combine by XOR, not OR.

4.1.3 Case: matching or flow - assigning work with quotas

Synthetic example: jobs A and B each need one worker. Worker Red can do either job and worker Blue can do only A; each worker can take at most one job. Can both jobs be assigned? Model jobs and individual workers as the two sides of a bipartite graph; an edge means the assignment is allowed. A matching selects edges with no shared endpoint. Here $A \to$ Blue and $B \to$ Red is a complete assignment.

For a standard one-to-one case, build adjacency list `g[job]` of worker indices, initialize the right-match array to -1, and call `hopcroftKarp(g, rightMatch)`. Its returned size is the largest number of jobs that can be assigned. It runs in $O(E\sqrt{V})$, where E is the number of allowed pairs and V the number of vertices; each job and worker must be used at most once, and the objective is unweighted cardinality. Use this direct helper when those constraints exactly match the story.

Now extend the example: Red is a team with two workers, both able to do A or B; Blue still has one worker who can do only A. A team node with capacity one would incorrectly restrict Red to one job. Model source s, job nodes, team nodes, and sink t. Add $s \to$ each job with capacity 1; each allowed job-to-team edge has capacity 1; add Red-to-t with capacity 2 and Blue-to-t with capacity 1. A unit of integral flow along $s \to job \to team \to t$ is one assignment. The invariant is flow conservation at every job/team node, plus edge capacities; integer capacities guarantee an integral maximum flow. Run Dinic from s to t. Both source edges saturated means both jobs were assigned.

Send B to Red and A to Blue, for flow value 2 and a complete assignment. If Blue is removed, Red's two workers can still take both jobs. This shows why a team quota belongs on the team-to-sink edge: source-to-job edges model job demand, while team-to-sink edges model available worker capacity.

Use the supplied `Dinic f(n); f.addEdge(u,v,cap); f.calc(s,t)` helper when capacities or shared bottlenecks matter. Its flow cost depends on the network and capacities; for the unit-capacity bipartite construction the specialized Hopcroft–Karp bound is $O(E\sqrt{V})$ and is usually the simpler guarantee. If preferences have costs, use min-cost flow rather than plain maximum flow. If each person may receive several jobs, increase that worker-to-sink capacity; if a team quota has a lower bound, ordinary max flow alone does not enforce it. A weighted matching helper may fit a dense assignment with one-to-one capacities, but the flow model is more adaptable when quotas or intermediate resource limits appear.

Practice: remove worker Blue from the two-job example. What is the maximum assignment size? Answer: 2. Both jobs can go to Red's two workers.

4.1.4 Case: capacity - binary search plus a greedy proof

Synthetic scheduling task: jobs arrive in a fixed order with nonnegative durations. Split them into at most k nonempty contiguous groups, and minimize the largest group sum. Jobs cannot be reordered or split. Example durations are $[4, 2, 3, 5]$ with $k = 2$. First define the decision question: for capacity C, can the jobs be assigned to at most k groups so every group sum is at most C? Increasing C can only make a previously feasible grouping remain feasible, so the predicate has the form false, false, ..., true, true.

For a fixed C, scan left to right and append each job to the current group if it fits; otherwise close that group and start the next with the job. This greedy scan uses the fewest possible groups: its first group consumes the longest prefix whose sum is at most C. Any feasible partition's first group ends no later; remove the greedy prefix and repeat the same argument on the suffix. Thus if greedy needs at most k groups, a valid partition exists; if it needs more, no partition can do better. The invariant during the scan is that the current group is the maximal feasible prefix not yet closed and the group count is the minimum needed for the processed prefix.

Use the largest job as the lower bound: any feasible capacity must be at least this large, though the bound itself need not be infeasible. The sum of all jobs is a feasible upper bound (one group always fits). Call `finals_cases::min_contiguous_capacity(a, groups)`; it assumes nonnegative jobs, a sum fitting in `long long`, and returns 0 for an empty list. The helper finds the minimum feasible integer capacity by monotone binary search and the greedy test. With n jobs and total sum S, this is $O(n \log(S + 1))$ time and $O(1)$ extra space, aside from the input.

Trace the example. At $C = 7$, greedy forms $[4, 2]$, then $[3]$, then $[5]$: three groups, so infeasible. At $C = 8$, it forms $[4, 2]$ and $[3, 5]$: two groups, so feasible. Binary search narrows the integer boundary between these cases; 8 is the minimum. A tempting wrong predicate is "greedy uses exactly k groups." It can become true and later false as capacity grows, so it is not the monotone decision question. For $[2, 2, 2]$ and $k = 2$, it uses three groups at $C = 2$, exactly two at $C = 4$, and one at $C = 6$; this false/true/false sequence cannot be binary-searched as a first-true test.

The local helper avoids a common midpoint bug by using `lo + (hi-lo)/2`. Its half-open search discards tested infeasible capacities; the original upper bound is a proven feasible fallback. The lower bound itself need not be infeasible. The exact API returns the minimum capacity, so the caller does not need to reproduce the search. If groups must be exactly k, at-most-k feasibility needs an extra proof: with nonnegative jobs, any group can be split until there are k groups provided $k \le n$ and empty groups are forbidden; this follows by splitting multi-job groups without increasing their sums. If negative durations are allowed, this greedy prefix proof fails: extending a group can reduce its sum, so taking the longest feasible prefix need not minimize the group count. Feasibility itself remains monotone in C: every partition feasible at C remains feasible at a larger capacity. If jobs may be reordered, sorting or a different partition model may be needed. If the objective is minimizing the largest number of jobs rather than total weight, the feasibility test changes even though binary search may still apply.

Practice: durations $[6, 1, 4]$, at most two groups. Is capacity 5 feasible? Answer: no; the first job alone exceeds 5. Capacity 6 is feasible with $[6]$, $[1, 4]$, so the minimum is 6.

CapacitySearch.h

Description: Minimum capacity for contiguous nonnegative jobs.

"Prelude.h" 01b1c4, 31 lines

```
namespace finals_cases {
// At most groups nonempty blocks; sum(a) fits ll; groups>0.
inline ll min_contiguous_capacity(const vector<ll> &a,
                                  int groups) {
  assert(groups > 0);
  if (a.empty()) return 0;
  ll lo = 0, hi = 0;
  for (ll x : a) {
    assert(x >= 0);
    lo = max(lo, x);
    hi += x;
  }
  auto fits = [&](ll capacity) {
    int used = 1;
    ll load = 0;
    for (ll x : a) {
      if (x > capacity) return false;
      if (load > capacity - x) {
        ++used;
        load = 0;
      }
      if (used > groups) return false;
      load += x;
    }
    return true;
  };
  // hi is proven feasible even though first_true never tests
  // it.
  return finals_core::first_true(lo, hi, fits);
}
} // namespace finals_cases
```

More states and tree reductions (5)

Prelude.h

Description: Shared includes, aliases and bounds. Include before the modules.

<algorithm>, <cstdint>, <limits>, <optional>, <stdexcept>, <string>, <vector> 14c483, 6 lines

```
namespace finals_dp_extra {
using namespace std;
using i64 = long long;

} // namespace finals_dp_extra
```

5.1 Three reusable DP and tree patterns

These small models cover different shapes of dependency: a subset of completed tasks, a parent-child choice, and a contiguous subtree range. First identify which previous decisions can affect the next one. If that information is a subset, use subset DP; if choices only interact across tree edges, root the tree; if every query asks about descendants, flatten the tree before selecting a range structure.

5.1.1 Order every item once: subset Hamiltonian path

Suppose each ordered pair of distinct jobs has a transition cost, and all jobs must be done once with the least total transition cost. A greedy "take the cheapest next job" can block a good continuation. For example, with costs $0 \to 1 = 1$, $0 \to 2 = 2$, $1 \to 2 = 100$, $2 \to 1 = 1$, and both edges back to 0 costing 100, greedy from 0 chooses 1 and pays 101; the order $0, 2, 1$ costs 3. The missing information is which jobs have already been used.

Define $dp[S][v]$ as the least cost of an ordering of exactly the set S, ending at v. Its last transition comes from a path on $S \setminus \{v\}$ ending at some u:

$$dp[S][v] = \min_{u \in S \setminus \{v\}} (dp[S \setminus \{v\}][u] + c[u][v]).$$

For a singleton, $dp[\{v\}][v] = 0$: the start is unrestricted and no edge has yet been paid. The answer is $\min_v dp[All][v]$, since the endpoint is also unrestricted. On the example, $dp[\{0, 2\}][2] = 2$, then $dp[All][1] = 3$. This is the Held–Karp subset recurrence: $O(n^2 2^n)$ time and $O(n2^n)$ memory. Store predecessor vertices if the order is needed. The tested helper returns only the cost and enforces $1 \le n \le 18$.

SubsetOrder.h

Description: SubsetOrder.

"Prelude.h" 0b9453, 34 lines

```
namespace finals_dp_extra {
// Minimum-cost Hamiltonian path in a complete directed matrix.
// n <= 18, costs in [0, 10^12]. O(n^2 2^n) time,
// O(n 2^n) memory. A path may start and end anywhere.
inline optional<i64>
hamiltonian_path(const vector<vector<i64>> &c) {
  int n = int(c.size());
  if (n == 0 || n > 18)
    throw invalid_argument("need 1..18 vertices");
  for (const auto &row : c) {
    if (int(row.size()) != n)
      throw invalid_argument("matrix must be square");
    for (i64 x : row)
```

```
      if (x < 0 || x > 1000000000000LL)
        throw invalid_argument("cost outside [0, 10^12]");
  }
  int count = 1 << n;
  const i64 inf = numeric_limits<i64>::max() / 4;
  vector<vector<i64>> dp(count, vector<i64>(n, inf));
  for (int v = 0; v < n; ++v) dp[1 << v][v] = 0;
  for (int mask = 1; mask < count; ++mask)
    for (int v = 0; v < n; ++v)
      if (mask >> v & 1) {
        i64 cur = dp[mask][v];
        if (cur == inf) continue;
        for (int w = 0; w < n; ++w)
          if (!(mask >> w & 1)) {
            int next = mask | (1 << w);
            dp[next][w] = min(dp[next][w], cur + c[v][w]);
          }
      }
  return *min_element(dp.back().begin(), dp.back().end());
}
} // namespace finals_dp_extra
```

This model changes if the route must start at a fixed vertex (initialize only that singleton), must return to start (add the closing edge), or may skip jobs (change the objective and permitted terminal masks; with nonnegative costs, freely skipping every job makes the optimum trivial). Precedence constraints require rejecting transitions that place a job before its prerequisites. For larger n, the exponential state count is the limit; use problem structure such as bitmask width, meet-in-the-middle, or a different optimization.

5.1.2 Pair disjoint endpoints in a tree

For a tree, maximize the number of edges chosen so no two share an endpoint. This is maximum matching, a useful starter when the statement says "pair people/items across links" and each endpoint can be used once. A global greedy choice is risky: a locally attractive edge can consume a vertex needed by multiple later choices. Root the tree; choices in different child subtrees become independent once we record whether the root is already paired with one child.

Let $free[u]$ be the best matching in the subtree when u is not matched to a child; let $best[u]$ allow either status. Then

$$free[u] = \sum_{vchild} best[v], \qquad best[u] = \max\left(free[u], \max_{vchild}\left(1 + free[v] + \sum_{w \neq v} best[w]\right)\right).$$

If u is paired with v, that edge contributes one and v cannot be paired below itself, which explains $free[v]$. For a root with two leaf children, $free = 0$ and $best = 1$: choose either edge. Each child is considered as the one partner, so the recurrence is linear in total tree degree. Supply symmetric adjacency for a connected simple tree. The implementation checks endpoints, arc count, cycles, and connectivity, and builds parent/order iteratively, then computes states in reverse order; this avoids recursion depth failures on a chain.

TreeMatching.h

Description: TreeMatching.

"Prelude.h" bfca68, 46 lines

```
namespace finals_dp_extra {
// Maximum number of pairwise vertex-disjoint edges in a tree.
// Requires symmetric adjacency of a simple undirected tree.
// Checks endpoints, arc count, traversal cycles/connectivity.
// O(n+m) time and memory.
inline int
maximum_tree_matching(const vector<vector<int>> &g) {
  int n = int(g.size());
  if (!n) throw invalid_argument("tree must be nonempty");
  long long arcs = 0;
  for (int u = 0; u < n; ++u) {
    arcs += g[u].size();
    for (int v : g[u])
      if (v < 0 || v >= n || v == u)
        throw invalid_argument("invalid tree edge");
  }
  if (arcs != 2LL * (n - 1))
    throw invalid_argument("not a tree");
  vector<int> par(n, -2), order{0};
  par[0] = -1;
  for (int i = 0; i < int(order.size()); ++i) {
    int u = order[i];
    for (int v : g[u])
      if (v != par[u]) {
        if (par[v] != -2) throw invalid_argument("not a tree");
        par[v] = u;
        order.push_back(v);
      }
  }
  if (int(order.size()) != n)
    throw invalid_argument("tree is disconnected");
  vector<int> free(n), used(n);
  for (int i = n - 1; i >= 0; --i) {
    int u = order[i], base = 0;
    for (int v : g[u])
      if (par[v] == u) base += max(free[v], used[v]);
    free[u] = base;
    used[u] = base;
    for (int v : g[u])
      if (par[v] == u)
        used[u] = max(used[u], base - max(free[v], used[v]) +
                                      free[v] + 1);
  }
  return max(free[0], used[0]);
}
} // namespace finals_dp_extra
```

This assumes every edge has equal value and the goal is cardinality. With edge weights, replace the added one with that edge's weight. If vertices have capacities above one, the two-state recurrence no longer captures their remaining capacity. If the graph has cycles, child subproblems are not separated by one parent edge; use a graph matching algorithm instead.

5.1.3 Turn descendants into a range

When many queries ask for the sum, count, minimum, or updates over a node's subtree, answering each by walking descendants can cost $O(n)$ per query. In a rooted tree, a preorder DFS visits each subtree consecutively. Record the position $tin[u]$ when u is appended, and $tout[u]$ as the exclusive end after all descendants. Then subtree u is exactly the half-open range $[tin[u], tout[u])$ in the flattened order. For example, root 0 with children 1 and 2, where 1 has child 3, may flatten to $[0, 1, 3, 2]$: subtree 1 is positions $[1, 3)$ and subtree 2 is $[3, 4)$.

SubtreeRanges.h

Description: SubtreeRanges.

"Prelude.h" f41048, 50 lines

```
namespace finals_dp_extra {
struct SubtreeRanges {
  vector<int> order, tin, tout;
};
// Iterative DFS flattening of a rooted tree at root. Subtree u
// is exactly order[tin[u]..tout[u]). Requires symmetric
// adjacency of a simple tree; checks
// endpoints/count/cycles/connectivity. O(n+m) time and memory;
// range works for Fenwick/segment trees.
inline SubtreeRanges
subtree_ranges(const vector<vector<int>> &g, int root = 0) {
  int n = int(g.size());
  if (!n || root < 0 || root >= n)
    throw invalid_argument("bad root/tree");
  long long arcs = 0;
  for (int u = 0; u < n; ++u) {
    arcs += g[u].size();
    for (int v : g[u])
      if (v < 0 || v >= n || v == u)
        throw invalid_argument("invalid tree edge");
  }
  if (arcs != 2LL * (n - 1))
    throw invalid_argument("not a tree");
  SubtreeRanges r;
  r.tin.resize(n);
  r.tout.resize(n);
  vector<int> par(n, -2), stack{root};
  par[root] = -1;
  while (!stack.empty()) {
    int u = stack.back();
    stack.pop_back();
    r.tin[u] = int(r.order.size());
    r.order.push_back(u);
    for (int v : g[u])
      if (v != par[u]) {
        if (par[v] != -2) throw invalid_argument("not a tree");
        par[v] = u;
        stack.push_back(v);
      }
  }
  if (int(r.order.size()) != n)
    throw invalid_argument("tree is disconnected");
  for (int u = 0; u < n; ++u) r.tout[u] = r.tin[u] + 1;
  for (int i = n - 1; i > 0; --i) {
    int u = r.order[i], p = par[u];
    r.tout[p] = max(r.tout[p], r.tout[u]);
  }
  return r;
}
} // namespace finals_dp_extra
```

Use a Fenwick tree for point updates and subtree sums: add a node's value at $tin[u]$, and query the flattened interval. Offline threshold counts can sort both events and nodes by value; activate nodes whose value meets the current threshold in a Fenwick tree over tin, then ask the subtree range sum. For example, if flattened values are $[5, 2, 7, 4]$, the query "count values at least 5 in subtree 1" activates positions 0 and 2 and returns the sum on $[1, 3)$, namely 1.

The flattening supports any range structure, but it does not make arbitrary path queries contiguous. Fixed-root subtree queries and path queries are different models. Re-rooting queries also change which vertices are descendants.

5.1.4 Find the kth object by skipping completion blocks

Synthetic task: list length-n binary strings with no adjacent ones in lexicographic order and return rank k, starting from rank 1. Leading zeroes are allowed. Do not generate all strings: count how many completions begin with 0, then decide whether to stay in that block or skip it.

Let $W[L]$ count valid strings of length L with no preceding 1. $W[0] = 1$, $W[1] = 2$, and $W[L] = W[L-1] + W[L-2]$: start with 0 and any valid suffix, or start with 10 and a shorter suffix. During reconstruction, a preceding 1 forces 0. Otherwise the 0-block has $W[L-1]$ completions. If k is at most that count, choose 0. Otherwise subtract the count and choose 1. Each prefix defines a contiguous block in lexicographic order.

For $n = 3$, the list is $000, 001, 010, 100, 101$. At rank 4, the 0-block has three entries, so skip it and choose 1 with new rank 1. Then 0 is forced; the final digit is 0. The answer is 100.

Unranking.h

Description: Unrank binary strings with no adjacent ones.

"Prelude.h" 68263e, 33 lines

```
namespace finals_dp_extra {
// 1-based rank among length-n strings, lexicographic 0
// before 1. Leading zeros allowed; 0<=n<=1000000, k>0. O(n)
// time/space. nullopt if fewer than k valid strings exist.
inline optional<string>
unrank_no_adjacent(int n, unsigned long long k) {
  if (n < 0 || n > 1000000 || k == 0)
    throw invalid_argument("unrank bounds");
  using U = unsigned long long;
  const U cap = k;
  vector<U> ways(n + 1, 1);
  if (n) ways[1] = min<U>(2, cap);
  for (int len = 2; len <= n; ++len) {
    U a = ways[len - 1], b = ways[len - 2];
    ways[len] = a >= cap - b ? cap : a + b;
  }
  if (ways[n] < k) return nullopt;
```

```
  string out;
  bool previous_one = false;
  for (int left = n; left; --left) {
    U zero_block = ways[left - 1];
    if (previous_one || k <= zero_block) {
      out += '0';
      previous_one = false;
    } else {
      k -= zero_block;
      out += '1';
      previous_one = true;
    }
  }
  return out;
}
} // namespace finals_dp_extra
```

Counts used for rank comparisons must not be reduced modulo a prime. The helper caps every count at the original requested rank using overflow-safe addition. A capped count still tells whether the current rank fits, because the current rank never increases. Cap at the original rank, not at a changing rank while building the table. The helper returns no answer for an invalid rank and handles $n = 0$ as one empty string.

For a different object, derive its completion-count state first. Ordering permutations by inversion count and then lexicographically, as in 2024 I, needs a different recurrence and proved bounds; this binary-string helper is not its solution. The reusable step is counting, skipping, and updating the residual rank. Test reconstruction by enumerating all small objects.

Adapt, test, and practice (6)

6.1 A budget turns subset search into DP

Synthetic task: each item has integer weight $w_i \geq 0$ and value v_i. Choose each item at most once, with total weight at most C, maximizing value. There are 2^n subsets. When nC fits the runtime and C fits memory, budget is a useful state dimension: all choices among the first i items with the same remaining capacity have the same future options.

Define $D_i[c]$ as the best value using the first i items with weight at most c. Empty selection is allowed, so $D_0[c] = 0$. Either skip item i, or take it from the previous row:

$$D_{i+1}[c] = \max\big(D_i[c], D_i[c - w_i] + v_i\big) \quad (c \geq w_i).$$

If it does not fit, only the skip term remains. This partitions every valid subset according to whether it contains item i, proving the transition. It costs $O(nC)$ rather than $O(2^n)$; huge binary-encoded capacities can still make it impractical. If the sum of values is small instead, consider minimum weight per value. A small number of items with huge weights may instead suggest meet-in-the-middle enumeration.

Trace weights $[2, 3, 4]$, values $[4, 5, 7]$, capacity 5. The first row after item 0 is $[0, 0, 4, 4, 4, 4]$. After item 1 it is $[0, 0, 4, 5, 5, 9]$; capacity 5 uses the previous value 4 at capacity 2 plus the new value 5. Item 2 improves capacity 4 to 7 but does not beat 9 at capacity 5. The answer is 9, choosing the first two items. The implementation book's `finals_core::knapsack01({2,3,4},{4,5,7},5)` computes this value.

Only the previous row is needed for values. In a single array, scan capacities downward so $D[c - w_i]$ still belongs to that previous row. For one item $(w, v) = (2, 3)$ and $C = 4$, scanning upward writes $D[2] = 3$, then uses it again to write $D[4] = 6$. That is two copies of one item and is wrong for 0–1 selection. It is exactly the intended behavior for an unbounded positive-weight item, where ascending order allows reuse.

If the total weight must equal C, initialize only $D[0] = 0$ and all other states as unreachable. Skip transitions out of unreachable states; an at-most initialization would incorrectly treat unattainable weights as free. To recover item indices, retain the rows and walk backward through skip/take choices. The supplied one-row helper returns only the optimal value.

Practice: weights $[2, 4]$, values $[3, 8]$, exact weight 3. Answer: unreachable. For weight at most 3 the answer is 3. This single change in the output constraint changes the base cases, even though the recurrence looks similar.

6.2 Turn a worked case into a new solution

Use the examples as derivations, not text-matching rules. For a new task, write down these differences before reusing code:

1. **Feasibility.** Are equal coordinates allowed? May vertices repeat? Must every item be selected or assigned? Are numbers positive?
2. **Objective.** Existence, count, minimum cost, lexicographically smallest answer, and reconstruction need different stored information.
3. **Scale.** Count the reduced states using the new bounds. Remainder DP is useful only if length times modulus is affordable.
4. **Timing and order.** Are updates online or offline? Are equal keys simultaneous? Can a hazard wait? Are dependencies acyclic?
5. **Output.** Does the helper return a count, parent indices, a value, or an optional answer? Translate it into the statement's format.

6.2.1 How to move from the naive recurrence

Start with a mathematically correct slow solution on small inputs. Then look at the operation inside its innermost loop:

- A contiguous sum: prefix sums remove the inner loop.
- Best earlier value with coordinate below x: process one coordinate in order and query a prefix maximum in the other. Preserve strict ties.

- Repeating the same suffix after different histories: memoize a state that retains exactly the history affecting that suffix.
- Rechecking every capacity: prove feasibility monotone, then search its boundary. The feasibility test itself still needs to be efficient.
- Trying all pairs of large objects: count equal features or contributions. Do not assume grouping by one feature preserves every term.

Fundraising demonstrates the second change. Enigma demonstrates the third. Displaying Decimals shows why deriving the value for one pair does not by itself solve aggregation over all pairs.

6.3 What to test before trusting the adaptation

Choose tests that attack the reduction. A random test of the final numeric answer does not explain why it is correct, but disagreement with a slow oracle immediately disproves it.

For each example below, write an enumerator that directly follows the task's rules, without reusing the optimized recurrence:

- Donation chains: enumerate subsets and check every selected pair's compatibility. This catches illegal ties and duplicate handling.
- Contiguous grouping: enumerate all cuts between adjacent jobs and minimize the largest group sum. This catches an incorrect greedy predicate.
- Toggle actions: BFS over board states with one click per edge. Compare the shortest distance with action-subset enumeration.
- Hazard escape: advance all possible monster positions and valid player positions one time layer at a time. Compare existence and path length; then replay the returned path for walls and unsafe simultaneous arrivals.
- Exact sum: enumerate every pair of endpoints and add its contents. This catches prefix initialization and query-before-insert mistakes.

These are the independent oracles used for this edition's five new modules. They exercise small cases; numeric bounds and maximum-size complexity still need separate checks. Named contest snippets have local test evidence, not new online-judge verdicts.

6.3.1 Counterexamples worth writing by hand

Equal keys; all values zero; one item; no solution; the start already at the goal; disconnected components; two different histories reaching the same vertex; an exact capacity boundary; a prefix sum repeated several times; a valid answer equal to the chosen sentinel. For each, predict the result before running the code. If the result is ambiguous, the model or output contract is not yet precise enough.

6.4 Practice without choosing another topic

Take one case at a time. Read its recognition and reduced task, cover the implementation, and write the state or invariant. Trace the given example. Implement it, compare small cases with the oracle, then answer the variation that changes one assumption. Finish by reopening a past statement without its algorithm name and explaining why the same method does or does not fit.

A concrete order for the team's reported gaps:

1. Signed prefix counts and sorted interval union. Make endpoints, strictness, and multiplicity reliable.
2. Parity-state BFS, then the hazard deadline model. Draw states and transitions before writing queue code.
3. Enigma. Derive the state, backward feasibility, and forward smallest reconstruction; do not merely copy the three loops.
4. Hidden Number and capacity search. Give the greedy proof in words, then deliberately change positivity or ordering and see what fails.
5. Fundraising. Implement the quadratic recurrence first, then replace its predecessor scan with a range maximum and test tied coordinates.
6. Fuzzy Factorization and single-fraction decimals. Separate exact representation from approximation and prove the chosen error convention.
7. One assignment/flow example. Name vertices, edges, capacities, and what one unit of flow represents before choosing the routine.

6.5 When the current pattern is insufficient

The original KACTL has additional specialists. Choose them only after the model demands their extra capability. The following are navigation notes, not complete implementations in this workbook:

- Dynamic tree path aggregates: heavy-light decomposition plus a range structure. Edge versus vertex values and update composition matter.
- Offline queries with reversible local changes: Mo ordering. You must supply correct add/remove operations; it is not a ready-made answer.
- A recurrence that sums products across all offsets: convolution via FFT/NTT. Check transform length, modulus, and coefficient bounds first.
- Many substring-order queries: suffix array/LCP. KMP and Z solve prefix/matching tasks, not arbitrary suffix ordering.
- Weighted assignment: min-cost flow or an assignment algorithm. Cardinality matching ignores weights; ordinary max flow ignores costs.

- Large integer factorization: algorithms beyond trial division. A factorization routine still does not replace a reduction that avoids factoring the original giant number.

A novel problem may combine several models or require a new proof. When a case's assumptions do not fit, return to the naive formulation and find the missing state or operation. Adding more templates without identifying that mismatch will not repair the solution.